

Proving Open Conjectures

Robert Veroff
Department of Computer Science
University of New Mexico
`www.cs.unm.edu/~veroff`

AITP 2026
Aug 30 –Sep 4, Aussois, France

Saturation-based Theorem Proving

- Clauses
- Proof by contradiction
- Inference rules
 - Resolution
 - Paramodulation (equality substitution)
- Main loop

```
while (no proof found)
{
    select "given clause" g
    apply inference rules to g and selected clauses
}
```

Other Things

- Process inferred clauses
 - Rewriting (changes clauses)
 - Subsumption (redundancy)
 - Attribute assignment
- Manage the clause space
 - Constraints on inference rules
 - Limits on what is kept (deletion strategies)

Our Focus

In this talk, we are focusing on systems and methods for proving specific challenge problems, for example, an open question posed by a research mathematician.

This is in contrast to the general development of automated systems.

Happiness is ...

- Proving the current target theorem.

- Proving a larger percentage of theorems in a test set.

Working on a Challenge Problem

- Original approach: Rerun the prover with different settings.
 - No significant progress between runs.
 - Proof required a single uniform strategy.
- Refinements:
 - Mine old problems for settings that work well.
 - Prover can adjust some settings during a run.
- Still too restrictive for especially difficult problems!

A Different Approach

- Concede that we are not likely to get a proof from scratch in a single run.
- Instead, work toward an eventual solution with collaborating sequences of prover runs.
- Each run is intended to make progress toward the goal of finding a proof of the target theorem.
- Two stages
 - Runs designed to focus later searches.
 - Guided runs that lead to a proof of the target theorem.

Features

- More collaborating processes can be added at any time.
- Different parts of a proof can be found with different methods (including different term orderings).

This approach has allowed us to find very long proofs.

Key Observation

Proofs of a theorem $A \Rightarrow c$ often will share steps with proofs of $A \cup a \Rightarrow c$, where a is an extra assumption.

A proof of $A \cup a \Rightarrow c$ is a rough sketch of a *possible* proof $A \Rightarrow c$.

This is the property that we try and leverage in our search.

Proof Sketches

Consider a derivation as a sequence of clauses,

$$c_1, c_2, \dots, c_i, \dots, c_j, \dots, c_n$$

where

- c_i is an extra assumption for the target theory A
- derived clause c_j has c_i in its derivation history

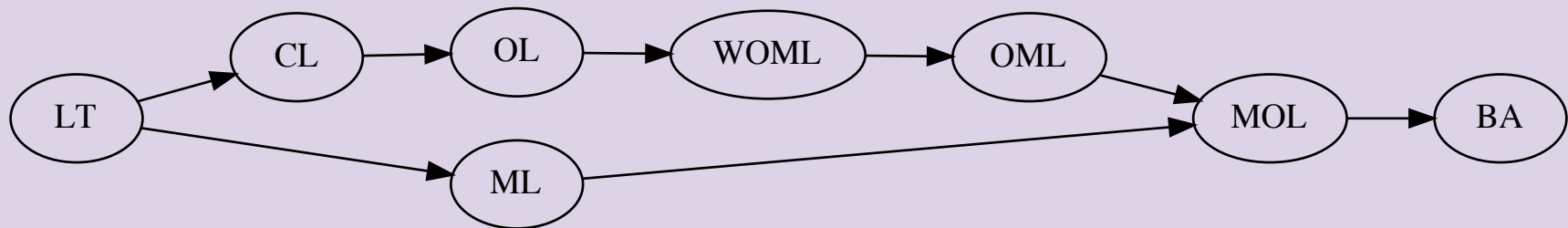
c_j either is derivable from A or it is not.

- if yes, it suffices to find a new derivation of c_j
- if no, it suffices to “bridge the gaps” to the consequences of c_j

In either case, we have a partial proof that *might* be easier to complete than finding a proof from scratch.

Theory Hierarchies

Example: Lattice Theory



$\mathcal{LT}$

- + Invertibility ($\mathcal{CL}$)
- + Compatibility ($\mathcal{OL}$)
- + Weak Orthomodularity ($\mathcal{WOML}$)
- + Orthomodularity ($\mathcal{OML}$)
- + Modularity ($\mathcal{MOL}$)
- + Distributivity ($\mathcal{BA}$)

Using Proof Sketches in Prover9

- Include previous relevant proofs as *hints*.
 - Hints are for guidance only, not inferences.
- Add a selection bias for picking *hint matchers* as given clauses.
 - A clause c matches a hint h if h is subsumable by c .
- Hint matchers can be included as lemmas in later runs of the same problem.
- Inferring and then assuming hint matchers is our working notion of making progress.

Proof Strategy

- Stage 1: Collect hints for the target theorem, for example, from proofs of the theorem with extra assumptions.
- Stage 2: Run the prover on the target theorem, noting the derivation of hint matchers. Repeat, including all previous hint matchers as lemmas.

Several Stage 2 invocations of the prover can run simultaneously. Each either completes a proof of the target theorem or collects new hint matchers to pass on to later runs.

A significant consequence is that different parts of a proof can be found with different search methods.

Note that Stage 2 does not need to be specified in advance. We can start additional searches at any time.

Stage 2 Iterations

Two layers of control (for historical reasons)

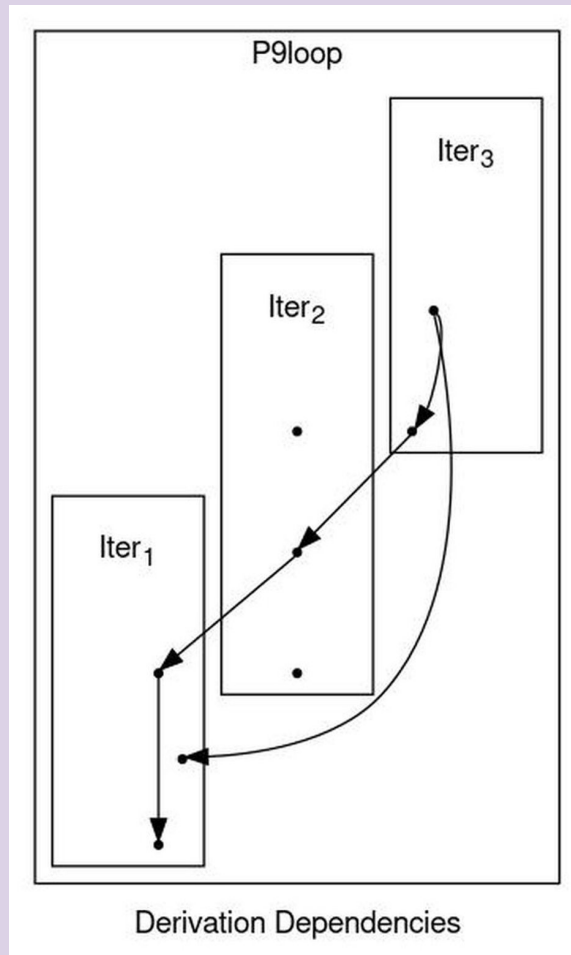
- A fixed sequence of Prover9 runs, one at a time (“P9loop”)
- Multiple P9loop executions run simultaneously (“Bigloop”)

Additional P9loop sequences can be specified and initiated at any time.

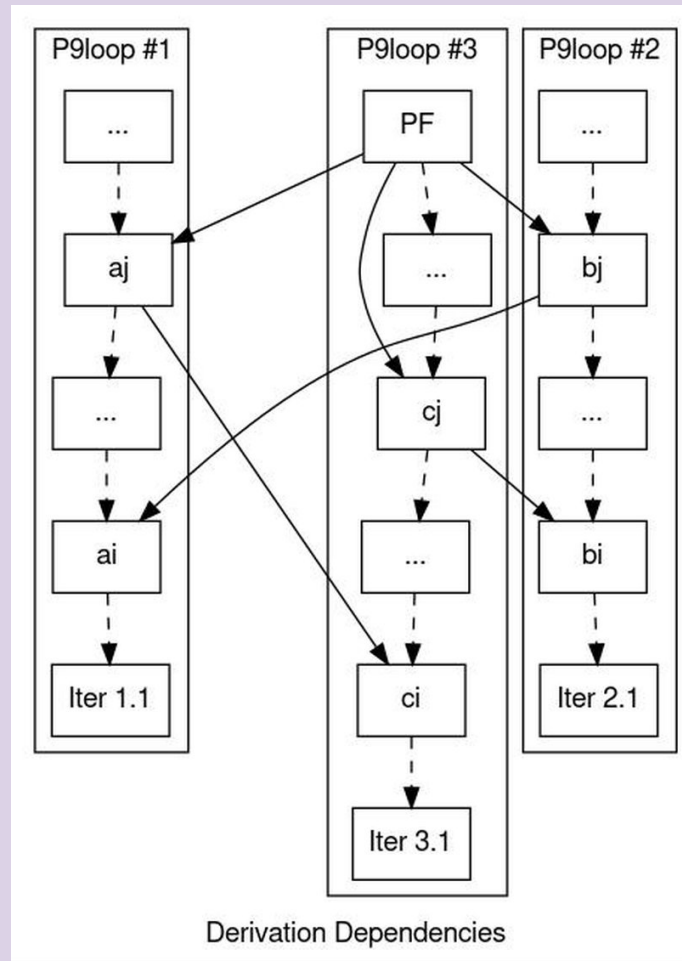
At the end of a Prover9 run, all hint matchers are collected from that run and included as lemmas in all later starts of Prover9.

Extracting hint matchers and starting the next Prover9 runs is automated.

P9loop: A Sequence of Prover9 Runs



Bigloop: Simultaneous Executions of P9loop



Extracting and Confirming a Proof

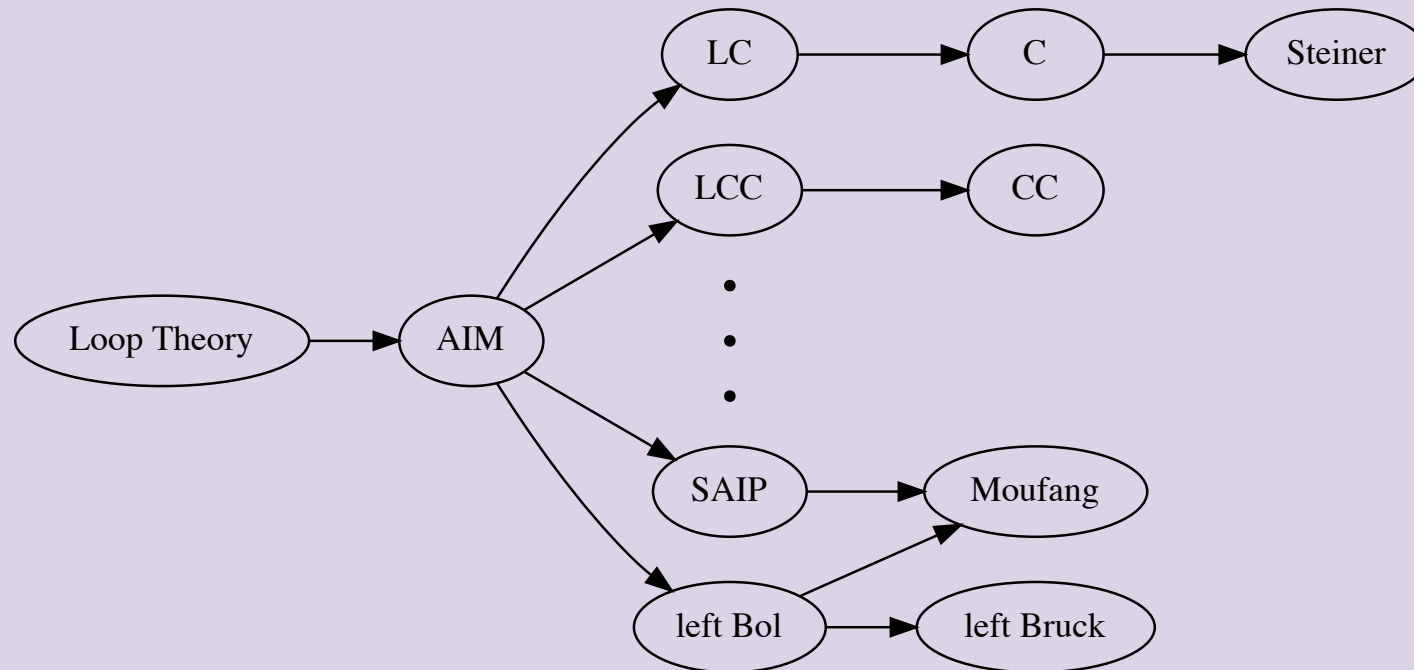
Dependencies in the network are followed level by level.

Proof is confirmed in a single run from the original input using the extracted (or partially extracted) proof as hints.

AIM Project

- Concerns Abelian inner mappings in loop theory
- Proposed as an automated deduction challenge by Michael Kinyon at the ADAM 2009 Workshop
- Collaboration with J. D. Phillips and Petr Vojtěchovský
- Several open questions in several loop varieties

The AIM Hierarchy



Several varieties of mathematical (not just strategic) interest.

AIM Problem

Strong Conjecture: Let Q be an AIM loop with nucleus $N(Q)$ and center $Z(Q)$. Then $Q/N(Q)$ is an abelian group, and $Q/Z(Q)$ is a group.

- Still open.
- Prover9 proofs for several previously open special cases of interest.

Weak Conjecture: AIM loops are nilpotent of class 3.

- Proved with a high-level mathematical argument.
- Still an ATP challenge.

`www.cs.unm.edu/~veroff/AIM_REDONE/`

Loop Axioms

```
% loop axioms  
1 * x = x.  
x * 1 = x.  
x \ (x * y) = y.  
x * (x \ y) = y.  
(x * y) / y = x.  
(x / y) * y = x.
```

Definitions

% associator (right associated * a = left associated)
 $(x * (y * z)) \setminus ((x * y) * z) = a(x, y, z) .$

% commutator ($(x * y) * K(y, x) = y * x$)
 $(x * y) \setminus (y * x) = K(y, x) .$

% inner mappings

$(y * x) \setminus (y * (x * u)) = L(u, x, y) .$
 $((u * x) * y) / (x * y) = R(u, x, y) .$
 $x \setminus (u * x) = T(u, x) .$

AIM Properties

```
% abelian inner mapping group
T(T(u,x),y) = T(T(u,y),x) .
T(L(u,x,y),z) = L(T(u,z),x,y) .
T(R(u,x,y),z) = R(T(u,z),x,y) .
L(R(u,x,y),z,w) = R(L(u,z,w),x,y) .
L(L(u,x,y),z,w) = L(L(u,z,w),x,y) .
R(R(u,x,y),z,w) = R(R(u,z,w),x,y) .
```

Main Conjecture

```
% aK ("single-a") goals
K(a(x,y,z),u) = 1      # label("Ka").
a(K(x,y),z,u) = 1      # label("aK1").
a(x,K(y,z),u) = 1      # label("aK2").
a(x,y,K(z,u)) = 1      # label("aK3").

% aa ("double-a") goals
a(a(x,y,z),u,w) = 1    # label("aa1").
a(x,a(y,z,u),w) = 1    # label("aa2").
a(x,y,a(z,u,w)) = 1    # label("aa3").
```

Example Nilpotency Class 3 Conditions

$$K(x, K(y, K(z, u))) = 1 \quad \% (3, 0)$$

$$K(K(x, a(y, z, u)), w) = 1 \quad \% (2, 1)$$

$$K(a(a(x, y, z), u, w), v5) = 1 \quad \% (1, 2)$$

$$a(a(a(x, y, z), u, w), v5, v6) = 1 \quad \% (0, 3)$$

Commutative AIM is NIL3

- Bigloop: 3 P9loop executions started
- Last Prover9 output file (proof found)
 - Length of proof: 3594 steps
 - 1155 assumptions in the proof are Bigloop lemmas
 - 60 Prover output files completed, potentially contributing lemmas
 - 25 of 60 contributed at least one lemma to the proof
 - Bigloop started: Thu Aug 7 02:25:46 2025
 - Proof found: Tue Sep 16 03:20:12 2025
- Single run proof based on extracted proof
 - Length of proof: 30835
 - Run started: Fri Oct 17 01:13:22 2025
 - Run ended: Fri Oct 17 09:15:10 2025

Nuclear Squares

$$\begin{aligned}(x * x) * (y * z) &= ((x * x) * y) * z \quad \% \text{ NucSq_l} \\ (x * (y * y)) * z &= x * ((y * y) * z) \quad \% \text{ NucSq_m} \\ (x * y) * (z * z) &= x * (y * (z * z)) \quad \% \text{ NucSq_r}\end{aligned}$$

Recent result: NucSq_r alone suffices for the strong AIM conjecture.

Bigloop proof: One of the goals of the strong conjecture was the key to completing the proof.

Bigloop Summary

- Bigloop: 22 P9loop executions started
- Last Prover9 output file (proof found)
 - Length of proof: 6929 steps
 - 3657 assumptions in the proof are Bigloop lemmas
 - 114 Prover output files completed, potentially contributing lemmas
 - 95 of 114 contributed at least one lemma to the proof
 - Bigloop started: Mon May 18 12:04:10 2026
 - Proof found: Tue May 19 20:00:56 2026
- Single run proof based on extracted proof
 - Length of proof: 265653
 - Run started: Mon Jun 1 12:30:42 2026
 - Run ended: Sun Jun 7 20:01:55 2026

Nice, but ...

Proofs are getting longer and longer.

Probably in part because the problems are getting more difficult.

But is our approach biased toward finding longer and longer proofs?

Thank you!